

1. Historias que requieren obligatoriamente persistencia

Las siguientes historias **dependen directamente de almacenamiento permanente en base de datos**:

- Registro de usuario
- Inicio de sesión
- Crear rutina personalizada
- Agregar ejercicios a una rutina
- Registrar peso y repeticiones
- Registrar tiempos de descanso
- Consultar historial de entrenamientos
- Seguimiento de cardio
- Editar rutina
- Eliminar rutina
- Notificaciones informativas (si son persistentes o programadas)

Estas historias son coherentes entre sí y construyen un modelo de datos consistente.

2. Coherencia entre historias y modelo de datos implícito

A partir de las historias se infiere el siguiente modelo lógico mínimo:

- **Usuario**
- **Rutina**
- **Ejercicio**
- **Sesion**
- **Set / RegistroEjercicio**
- **RegistroCardio**
- **Notificacion**

Evaluación de consistencia

a) Registro e inicio de sesión

Coherente técnicamente.

Requiere:

- Tabla usuario
- Persistencia de credenciales cifradas

- Relación 1:N entre Usuario y Rutina

No hay contradicción con el resto de historias.

b) Crear / Editar / Eliminar rutina

Coherente y bien definida a nivel funcional.

Requiere:

- Tabla rutina
- FK hacia usuario
- Operaciones CRUD completas

Existe consistencia porque:

- Primero se crea
- Luego se edita
- Finalmente se elimina

Flujo lógico correcto.

c) Agregar ejercicios a una rutina

Coherente pero implica una relación intermedia:

Se necesita una tabla puente:

- rutina_ejercicio
 - rutina_id
 - ejercicio_id
 - orden
 - configuraciones (series, repeticiones base, descanso)

Sin esa tabla la historia no es técnicamente viable.

d) Registrar peso y repeticiones

Aquí aparece un punto clave.

Esta historia implica que **no basta con guardar la rutina**, sino que debe existir una entidad transaccional tipo:

- sesion_entrenamiento
- registro_ejercicio o set

Porque el peso y las repeticiones cambian en cada sesión.

Es coherente con:

- “Consultar historial”
- “Visualizar estadísticas”

Si no existe una entidad de sesión independiente, la historia sería técnicamente incoherente.

e) Registrar tiempos de descanso

Puede modelarse de dos formas:

1. Como atributo del ejercicio en la rutina (configuración base)
2. Como dato real registrado en cada sesión

Debe definirse claramente cuál es el enfoque.

Si no se especifica, hay ambigüedad técnica.

f) Consultar historial de entrenamientos

Coherente solo si existe:

- Tabla sesion_entrenamiento
- Relación con registros de sets
- Marca temporal (fecha)

Si no se almacenan sesiones con timestamp, esta historia no es posible.

g) Visualizar estadísticas

No requiere nueva persistencia si ya existen:

- Sesiones
- Sets
- Registros cardio

Se calcula sobre datos almacenados.

Es coherente y dependiente del historial.

h) Seguimiento de cardio

Implica otra entidad:

- registro_cardio
 - usuario_id
 - fecha
 - tiempo
 - distancia
 - intensidad

Coherente con historial y estadísticas.

i) Biblioteca de ejercicios

Puede ser:

- Datos estáticos precargados
o
- Tabla persistente ejercicio

Si los ejercicios son editables en el futuro, deben persistirse.

No hay contradicción técnica.

j) Notificaciones informativas

Si son:

- Push dinámicas → no requieren persistencia fuerte.
- Recordatorios programados → requieren tabla notificación.

La historia es ambigua y debería especificar eso.

3. Coherencia global

Las historias son técnicamente coherentes entre sí.

No hay contradicciones lógicas.

Construyen un flujo correcto:

1. Usuario se registra
2. Crea rutinas
3. Ejecuta sesiones
4. Registra datos
5. Consulta historial

6. Visualiza estadísticas

El problema no es incoherencia, sino **falta de especificación técnica** en:

- Definición de entidades explícitas
- Diferenciación entre configuración y registro real
- Definición clara de sesión de entrenamiento

4. Riesgos de incoherencia si no se modela bien

- Confundir “rutina” con “sesión ejecutada”
- Guardar peso y repeticiones en la rutina en vez de en una sesión
- No almacenar fecha/hora
- No definir claves foráneas
- No manejar eliminación en cascada

5. Conclusión

Las historias de usuario en /docs son funcionalmente consistentes y técnicamente viables con una arquitectura típica backend (Spring Boot + PostgreSQL).

Son coherentes siempre que:

- Se modele explícitamente la entidad Sesión
- Se separen configuraciones base de datos reales registrados
- Se definan relaciones claras entre Usuario, Rutina y Ejecución